EVENTS SECTION API CONTRACT

1. Club Management

1.1 Create Club (Admin)

Description: Creates a new club or entity profile. Media files must be uploaded to cloud storage by the client prior to calling this endpoint or we can send the file to backend for uploading(discuss this with vedant).

Endpoint: POST /api/v1/clubs/create-club

Content-Type: application/json

Request Payload:

JSON

```
{
 "clubName": "Consulting and Analytics",
 "clubLogo": "https://storage.googleapis.com/bucket/path/logo.png",
 "category": "Technical",
 "clubRoomLocation": "New SAC, Ground Floor, Left Side - Tech Board Room",
 "clubDescription": "Core 1, Library entrance, etc...",
 "howToJoin": "Process to join the club...",
 "boardName": "Technical Board",
 "achievements": [
  {
   "title": "Best Tech Club",
   "year": 2024
  }
 ],
 "team": [
  {
   "name": "Ayesha Singh",
   "email": "a.singh@iitg.ac.in",
   "position": "Web Design Head",
   "number": "+919876543210"
  }
 ]
}
```

Success Response (201 Created):

JSON
```
{
  "success": true,
  "data": { "club_id": "c_8f72a9b1" }
}
```

1.2 Get All Clubs / Fests

Description: Fetches a list of clubs, supporting category filters (e.g., Welfare, Technical, Cultural).

Endpoint: GET /api/v1/clubs/get-all

Query Parameters:

- category (optional) - e.g., Technical, Cultural
- search (optional) - String to search by club name.

Success Response (200 OK):

JSON
```
{
  "success": true,
  "data": [
    {
      "club_id": "c_8f72a9b1",
      "clubName": "Consulting and Analytics",
      "clubLogo": "https://storage.googleapis.com/bucket/path/logo.png",
      "boardName": "Technical Board"
    }
  ]
}
```

1.3 Get Club Details

Description: Fetches detailed view of a club, including its active team and recent events.
Endpoint: GET /api/v1/clubs/:club_id

Success Response (200 OK):

JSON
```
{
  "success": true,
  "data": {
```

```
  "_id": "c_8f72a9b1",
  "clubName": "Consulting and Analytics",
  "clubLogo": "...",
  "clubRoomLocation": "...",
  "clubDescription": "...",
  "howToJoin": "...",
  "achievements": [],
  "team": [],
  "recentEvents": [
   {
    "event_id": "evt_9b1a2c3d",
    "name": "Rangmunch - Inter Hostel Stand Up",
    "eventBanner": "..."
   }
  ]
 }
}
```


---

2. Event Management (Admin & Creation)

2.1 Create Event

Description: Creates a new event (can be saved as Draft or Posted active).

Endpoint: POST /api/v1/events/create-event

Content-Type: application/json

Headers: x-email: “enter emailId”

Request Payload:

JSON


```
{
 "club_id": "c_8f72a9b1",
 "name": "Rangmunch - Inter Hostel Stand Up",
 "status": "POSTED",
 "eventBanner": "https://storage.googleapis.com/bucket/path/banner.png",
 "date": "2025-09-02",
 "startTime": "2025-09-02T09:00:00.000Z",
 "endTime": "2025-09-02T12:00:00.000Z",
 "location": "Main Auditorium, Lecture Hall Complex",
 "tags": ["Finance", "Cultural"],
```

```
  "description": "Lorem voluptate id mollit ad enim...",
  "whoShouldAttend": "Students interested in standup...",
  "poc": [
   {
    "name": "Ayush Bahuguna",
    "email": "ayush@iitg.ac.in",
    "position": "Events Head",
    "number": "+911234567890"
   }
  ],
  "guests": [
   {
    "name": "Nikhil Kamat",
    "position": "Co Founder, Zerodha",
    "photo": "https://storage.googleapis.com/bucket/path/guest.jpg",
    "socials": []
   }
  ],
  "additionalLinks": []
}
```

(Note: Use status: "DRAFT" when using the "Save Draft" flow).

Success Response (201 Created):

JSON

```
{
  "success": true,
  "data": { "_id": "evt_9b1a2c3d" }
}
```

2.2 Update Event

Description: Edits an existing event.

Endpoint: PUT /api/v1/events/update-events/:event_id

Headers: x-email: “enter emailId”

Payload: Same as Create Event (partial updates supported).

2.3 Delete Event

Description: Deletes an event.

Headers: x-email: “enter emailId”

Endpoint: DELETE /api/v1/events/:event_id

2.4 Get Admin Events

Description: Fetches events managed by the admin's club (Drafts, Past Events, Upcoming).
Endpoint: GET /api/v1/events/get-events

Query Parameters:

- club_id (required)
- status (optional) - e.g., DRAFT, POSTED, PAST

---

3. Event Discovery & User Views

3.1 Get All Events (Feed & Filters)

Description: Fetches events for the main feed, calendar view, or filtered search.\

Endpoint: GET /api/v1/events

Query Parameters:

- feedType (optional) - TODAY, TRENDING, INTERESTS
- search (optional)
- date (optional) - ISO format YYYY-MM-DD
- tags (optional) - Comma-separated (e.g., Tech,Finance)
- limit (optional) - Default 20
- page (optional) - Default 1

Success Response (200 OK):

JSON


```
{
  "success": true,
  "data": [
    {
      "_id": "evt_9b1a2c3d",
      "name": "Rangmunch - Inter Hostel Stand Up",
      "eventBanner": "...",
      "startTime": "2025-09-02T09:00:00.000Z",
      "endTime": "2025-09-02T12:00:00.000Z",
```

```
    "location": "Main Auditorium",
    "tags": ["Finance", "Cultural"],
    "clubName": "Students' Web Committee",
    "stats": {
      "interested": 350,
      "going": 120
    },
    "userStatus": {
      "isInterested": true,
      "isRegistered": false
    }
  }
 ]
}
```


3.2 Get Single Event Details

Description: Popup view for a specific event.

Endpoint: GET /api/v1/event-discovery/get-event-details/:event_id *(Response matches the complete event object created in 2.1, plus stats and userStatus).*

3.3 Get User's Saved/Attended Events

Description: Fetches events liked by the user.

Endpoint: GET /api/v1/event-discovery/liked

Query Parameters:

- rollNo (required)
- email (required)

---

4. User Interactions

4.1 Like Event

Description: Marks or unmarks that a user like a particular event.

Endpoint: POST /api/v1/events/like-event/

Query Parameters:

- rollNo (required)
- email (required)
- event_id(required)

Body


```
{

    "like": false

}
```


Response:

JSON


```
{
    "success": true,
    "like": false,
    "interaction": {
        "_id": "6a7b07f78dc06ef961fbe6cd",
        "event_id": "6a12b154d4f128b3e5638592",
        "student_id": "6a7b07f78dc06ef961fbe6cc",
        "__v": 0,
        "createdAt": "2026-08-11T11:31:03.168Z",
        "like": false,
        "updatedAt": "2026-08-11T11:31:24.572Z"
    }
}
```


4.2 Submit Feedback

Description: Allows a user to leave feedback for a past/attended event.

Endpoint: POST /api/v1/students/submit-feedback

Query Parameters:

- rollNo (required)

- email (required)
- event_id(required)

Request Payload:

```
{
    "rating": 4, // { type: Number, min: 1, max: 5 },
    "comment": "Event was fun, really enjoyed it!",
    "suggestions": "you can give free pizza next time!!"
}
```

4.3 Get Event Feedback (Admin)

Description: Fetches paginated feedback comments for an event.

Endpoint: GET /api/v1/events/:event_id/feedback

Success Response:

JSON


```
{
    "success": true,
    "message": "Event feedback fetched successfully",
    "count": 1,
    "data": [
        {
            "feedback": {
                "comment": "Event was fun, really enjoyed it!",
                "rating": 4,
                "suggestions": "you can give free pizza next time!!"
            },
            "_id": "6a7b25688dc06ef961fbeb01",
            "event_id": "6a12b17ad4f128b3e5638596",
            "student_id": {
                "_id": "6a7b25688dc06ef961fbeb00",
                "email": "vedant.shah@iitg.ac.in"
            },
            "__v": 0,
            "createdAt": "2026-08-11T13:36:40.123Z",
            "like": true,
            "updatedAt": "2026-08-11T13:54:47.036Z"
```

```
            }
        ]
}
```

---

5. Global Error Handling

Validation failures (e.g., missing mandatory fields) or Auth errors will return a standard error structure.

Example (400 Bad Request(routes does not exist) / 403 Forbidden(not permitted to send the request)/500 Server error)

JSON

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "eventBanner and club_id are required fields."
  }
}
```